Week 3 – Interrupts, Debouncing, Scheduling & ADC

This lab is the second of three labs that help you complete your first project milestone (MS1) due in week 5.

1. Lab objectives

- Set up interrupts and work with interrupt handlers
- Use the debugging tools in STM32CubeIDE
- Generate a program which responds reliably to button pushes
- Configure the ADC peripheral and read its values
- Develop an intuition for code cleanliness and modularisation

2. Set up a pin-change interrupt

At this point, you should have completed all the exercises in the week 2 lab and **committed the associated code to version control.** Use your code from the end of week 2 as a starting point for this week's lab.

A pin-change interrupt is a hardware interrupt[1] that is triggered when the logic level on a GPIO pin changes from low to high (rising edge) and/or from high to low (falling edge). We will use SW3, connected to the pin PC10, to explore this concept.

1. Open the configuration file (.ioc) in STM32CubeMX.
2. Under Pinout & Configuration, click on PC10 (which should be labelled as SW3 from the previous lab), and change the configuration from "GPIO_Input" to "GPIO_EXTI10".[2]
3. Using the pane on the left, expand "System Core" and select "GPIO".
4. Repopulate the user label for PC10 with "SW3" and choose "Pull-down".
5. Leave the GPIO mode as the default value (external interrupt rising edge).
6. Open the NVIC tab and enable "EXTI line 4 to 15 interrupts".
7. Generate the new code.
8. In STM32CubeIDE, open app.c and comment out the existing code that handles reading the SW3 GPIO and turning on the corresponding LED (from last week).
9. Define the following function[3] at the bottom of the module:

```
void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)
{
        if (GPIO_Pin & GPIO_PIN_10)
        {
                rgb_led_toggle(RGB_RIGHT);
        }
}
```

[1] An interrupt causes normal program execution to be paused and an interrupt handler (i.e. a special function) to be executed. Often, the interrupt handler will itself call another "call back" function. Once the interrupt handler completes, normal program execution resumes at the instruction where it left off.
[2] EXTI10 refers to line #10 in the Extended Interrupt and Event Controller unit in the MCU. This is internally connected to PC10, and so can monitor pin-change events on it.
[3] This function is called by the generated driver code when the corresponding event is triggered. You can follow the chain of function calls that leads to **`HAL_GPIO_EXTI_Rising_Callback`** using the C/C++ search tool (Ctrl+H). This should take you to an interrupt handler defined in stm32c0xx_it.c. How is this handler called, you ask? Shameless plug to your interrupt lecture notes!

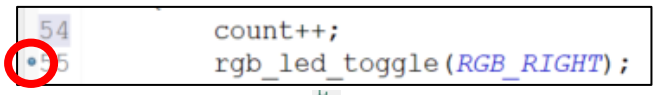
Note if you are not using the RGB library provided in the previous lab, you will need to find equivalent C code that toggles the correct LED.

Run your code and test it by pressing SW3 several times. You should find that this indeed toggles the right RGB LED. However, it seems to do so somewhat *inconsistently*. Why do you think this is the case?

3. Debugging

The desired behaviour is that every time the switch is pressed, the toggle function is called exactly once. But this doesn't seem to always be the case. Let's use the debugger to explore what is happening under the hood.

1. Declare a static integer variable "count" in the callback function.
2. Make it increment just before the LED toggle function is called.
3. Set a breakpoint by double clicking the strip to left of the line of code that follows the incrementation of the count variable. A blue dot should appear like so:



4. Press the debug button to load the code and start a debug session, then let the code run by pressing the resume button .
5. Press SW3 – the code should "break" (pause) where you have set the breakpoint.
6. Take note of the value of the variable "count" in the Variables tab in the top right corner of the IDE.
7. Press the play button to let the processor continue running the code. Pay careful attention to what happens (or doesn't happen) next.
8. Repeat steps 5-7 several times.

Have you noticed anything odd? Can you explain what might be causing the inconsistencies a little better?

Commit the changes you have made so far to save your progress. We will shortly revert these changes to apply a better implementation.

4. Button debouncing

Many buttons mechanically bounce up and down with a single press, resulting in a bounce in the corresponding electrical signal. Without debouncing logic, each bounce is erroneously interpreted as a separate button press. In this section, you will use debouncing code provided to you and test the difference for yourself.

Firstly, we will revert the latest commit you have made to undo what we have done on pin-change interrupts. **Close the IDE and MX software** then run the following command:

```
git revert --no-edit HEAD
```

This reverts the latest commit by creating a new commit that applies the opposite changes. This is the safest way to undo work in git as the reverted change is retained in commit history. Use the command '`git log`' to confirm the commit history is as expected.

You can now reopen the IDE and MX.

To use the debouncing code:

1. Download **buttons.c** and **buttons.h** from the lab folder on Learn.
2. Copy **buttons.c** to the *Src* subdirectory in your project folder, and **buttons.h** to the *Inc* subdirectory.
3. Scan the comments in these two files to understand how this module works.[4]
4. In app.c, include the **buttons.h** header file.
5. Make a function call to initialise the buttons module once, before the while loop.
6. Make a function call to update the buttons state at each iteration of the while loop.
7. Instead of directly reading the GPIO pin state for each button, use the module provided to check the button state.
8. Modify the code so that each button press simply toggles the corresponding LED.

Load and test the new code. You should find that only long button holds result in toggling the corresponding LED.

9. Gradually decrease the while loop delay period, testing between each iteration, until the buttons (specifically SW3 & SW4) work seamlessly. To optimise CPU usage, do not make the delay value any lower than it needs to be.

Commit & push!

5. Basic scheduling

You should now have button polling and LD1 blinking at the same rate. But what if we want LD1 to flash at a slower rate, e.g. every 500 ms or at 2 Hz?

Download **basic_scheduling_example.c** and open it in your favourite code editor. Do not add this file to your project as you will only use it as a reference to write your own code. Scan the code and answer the following questions:

1. What variable does HAL_GetTick() return? *Hint: use Ctrl+H in your IDE to run a project-level search for the function definition*.
2. Which function increments the variable returned by HAL_GetTick()? How is this function triggered?
3. How does the basic task scheduler provided know when a task is ready to be executed?

Complete Table 1 based on the delay period you determined in section 4.

Task	Function name	Frequency (Hz)
LD1 blinking	blinky_task_execute()	2
Button polling	button_task_execute()	

Table 1. Task details

Back to your code, refactor the button polling logic and the blinky logic, each into a separate function as labelled in the table above. Build and test that your code still works.

Follow the pattern in **basic_scheduling_example.c** to allow these functions to run at the correct frequencies. Use #defines and variable names that correctly correspond to the task names.

[4] The debouncing code provided is polling-based. However, debouncing can also be achieved with pin-change interrupts just as well.

You should be able to see LD1 blinking slowly and the buttons working seamlessly. Celebrate your achievement with a commit and a push!

6. Joystick ADC

There are two analogue signals that correspond to the joystick position: an X value and a Y value. Therefore, we will have to read two different ADC channels. To do this efficiently, we will use the direct memory access (DMA)[5] feature in the MCU.

Use the schematic to complete the following table:

User label	GPIO pin ID	ADC Channel
JOYSTICK_X		
JOYSTICK_Y		

To configure the ADC channels, do the following:

1. In MX, open the project configuration (.ioc) file.
2. In the pinout view, click on each of the joystick pins and select the corresponding ADC channel according to the table above.
3. Update the user labels for the ADC pins to follow the table above by right clicking each pin and selecting "Enter user label".
4. Select DMA under System Core in the left pane.
5. Click "Add" at the bottom of the window.
6. Under DMA request, select "ADC1".
7. Leave all the settings as default and generate the code.

To trigger the ADC and read the values, make the following code additions to app.c:

1. Include "adc.h" at the top of the file.
2. Declare a static array of two 16-bit unsigned integers, global to the module, to store the converted values into:

```
static uint16_t raw_adc[2];
```

3. Periodically trigger the ADC conversion process - you can add this directly under the call to the button task for now:

```
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)raw_adc, 2);
```

4. Finally, define an empty callback function that the HAL will call when the conversion is completed:[6]

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{

}
```

Since we do not have a display or a serial output yet (spoiler alert for next lab), we will use the debugger to test that we have configured the ADC peripheral correctly.

1. Set a breakpoint on the empty line of the callback function.

[5] DMA will be covered in detail in later lectures. For now, it is simply a way to directly transfer data between two locations of memory when an event occurs, without the involvement of the CPU. In the case of the ADC, this allows the ADC values to be stored directly to an address in memory (i.e. a variable) as soon as the conversion is completed – how neat!

[6] You can use the Ctrl+H trick to trace the call chain for this function.

2. Run the debugger.

The code should pause almost immediately. We want to inspect the "raw_adc" variable to see if this is updated with the correct values, but this is not visible in Variables tab as it is not declared inside the function.

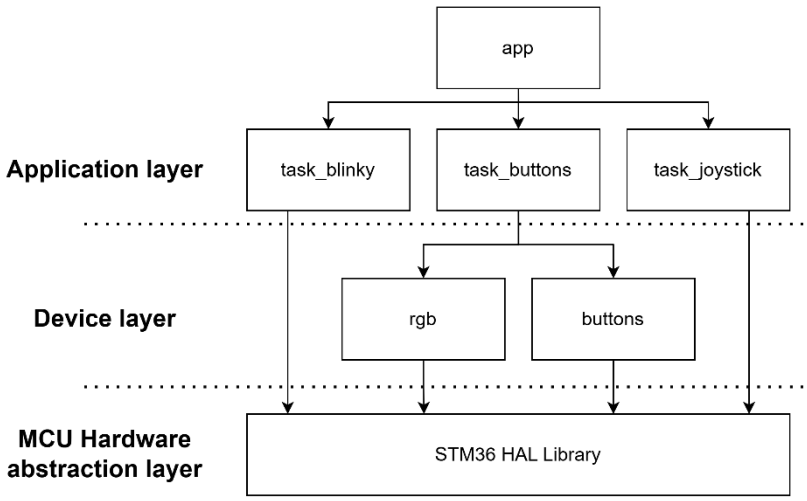
3. Open the "Expressions" tab on the top-right corner of the IDE window.
4. Click "Add a new expression" and type "raw_adc" (without the quotations).
5. Expand the array by clicking the arrow next to its name to see both values.
6. Click play to run the code again. It should pause again when the callback function is called next, signalling that another conversion has been completed.
7. Move the joystick into various position and repeat steps 5 & 6 to complete the table below:

ADC value	Joystick axis (X/Y)	Resting point value	Minimum value	Maximum value	Direction of increase (e.g. left to right)
raw_adc[0]					
raw_adc[1]					

Once you have confirmed that your code works (and committed the changes to version control), create a new joystick task with an appropriate frequency by following the same pattern as the other tasks.

7. Modularisation

As your code increases in complexity, modularisation becomes crucial to the maintainability of your code. Now that we have a task scheduler, an easy way to start modularising is to simply have a module for each of the tasks. A common pattern is to have an initialise and an execute functions for each module that contains a task.



Refactor your code to have a module for each task that you created. You should also move the static variables and other functions associated with each task **but not the task scheduler**. It is recommended to do this in small, testable increments so that you do not spend hours debugging. And of course, commit and push your changes when refactoring is complete.

Alongside the code provided, you should have a layered architectural pattern emerging as shown.[7] Take a moment to appreciate the work you have done so far!

[7] Some further improvements can be made. For example, the scheduling logic can be contained in its own module and driver modules can be written for blinky and joystick, so the corresponding tasks remain agnostic of the MCU. You can consider these ideas, alongside your own, for your final code submission.